Computational MR imaging
Laboratory 2: k-space sampling and Fourier reconstruction

Code submission is due by 23:55 on Monday. Please upload your code to StudOn in a described format. Late submissions will not be accepted.

Learning objectives

- Reconstruct Cartesian MRI data
- Compute the point spread function
- Apply Square and Hamming filter to k-space
- Understand oversampling along the Readout direction

1. FFT reconstruction of Cartesian MRI data

1.1 kdata included in the exercise

- Fully sampled kdata: *load_kdata()*
- Oversampled kdata: *load_kdata_os()*

1.2 Implement methods: *fft2c()* and *ifft2c()* to perform forward and inverse Fourier transforms for MRI data and reconstruct kdata

- Consider the shift of the zero-frequency component by using fftshift / ifftshift
- Keep in mind that you have to preserve signal energy between transforms
- Plot the magnitude and phase of the reconstructed image

2. Square filter on k-space

2.1 Implement method: *get_sqaure_filter()* to define a square filter, which is the same size as kdata, outer with 0s and center with 1s in different central sizes of 64×64, 128×128, and 256×256.

2.2 Implement method: *filtering()* that applies the square filter to the kdata

- Repeat this for [64x64, 128x128, 256x256] filter sizes.

2.3 Reconstruct all of the square-filtered k-space and compare them with the reconstructed image from Task 1

2.4 Discuss the effects of k-space truncation

3. Point spread function (PSF)

3.1 Implement a method: *get_psf_1d_square()* to generate 1D PSF for square filter

- Take the middle row of the generated 2D filter
- Think about how to use *ifft2c* method for the 1D signal.

3.2 Compute the (1D) PSF for each k-space truncation pattern defined in Task 2

3.3 Plot the different PSFs in the same figure using different colors

3.4 Implement method: *get_fwhm()* to compute the full-width at half-maximum (FWHM) for each PSF

3.5 Discuss the differences between different filter sizes in terms of spatial resolution

4. Effects of the Hamming filter on zero-padded k-space

4.1 Implement method: *get_hamming_filter()* to define a Hamming filter, which is the same size as kdata. It is similar to square filter but the center values follow bell shaped curves.

- Define the Hamming filter with shape: [64×64, 128×128, and 256×256]

4.2 Reconstruct all of the Hamming-filtered k-space and compare them with the reconstructed image from Task 1 and Task 2

4.3 Compute PSFs for different patterns and compare them to the PSFs from Task 3

4.4 Compute FWHMs and compare them to the values from Task 3

4.5 Discuss advantages and disadvantages of different windows with respect to each other, in particular the effects on resolution and Gibbs ringing.

5. Oversampling the readout dimension

5.1 Reconstruct and remove the oversampling by cropping the frequency-encoding dimension in image domain in method: *cut_oversampled()*

- The frequency-encoding dimension is oversampled by a factor of 2

5.2 Discuss the effect of oversampling